



Trabajo Práctico

Optimización de Decisiones Secuenciales Bajo Incertidumbre

Master in Management & Analytics -
Universidad Torcuato Di Tella

Julio 2025

Echavarria Agote, Pedro

Melnitzky, Eitán

Suffern, Dominique

1. Descripción del problema

La inyección de agua es una estrategia ampliamente utilizada para mantener la presión del yacimiento y desplazar el petróleo hacia los pozos productores. Su implementación requiere decisiones operativas que deben equilibrar el volumen de inyección, los costos de operación y disposición de agua y el impacto en la calidad de los fluidos producidos. Una inyección excesiva puede acelerar el aumento del watercut, lo que incrementa los costos y reduce la eficiencia de recuperación. Por el contrario, una inyección insuficiente puede causar pérdida de presión y menor producción de petróleo.

Este modelo utiliza dos variables clave para representar el estado del sistema. El watercut indica la proporción de agua en la producción y permite capturar el efecto de las decisiones de inyección sobre la calidad del fluido producido. La producción acumulada mide el nivel de depleción del yacimiento y resume de forma indirecta el historial productivo y las condiciones del reservorio.

Aunque la producción acumulada se actualiza en función del watercut y de las decisiones tomadas, se mantiene como variable de estado porque resume la historia completa de producción del sistema. Esto es importante dado que de un yacimiento que ha sido intensivamente producido se espera que su potencial de explotación sea menor que otro yacimiento con el mismo wcut pero menor historia de producción.

Para el problema a atacar, se han definido las siguientes variables para modelar la realidad de la situación. En el mismo se tomaron algunas simplificaciones:

- Se asume una operación lejos de la saturación de petróleo residual (SOR).
- No se modelan propiedades petrofísicas ni presiones explícitamente, estas se ven incluidas indirectamente en el NP.
- Las facilities permiten ajustar la inyección sin restricciones técnicas.
- Las tasas de inyección se mantienen constantes durante cada trimestre.
- El precio del petróleo, los costos y la tasa de descuento se asumen constantes en el tiempo.
- No se consideran eventos de fallas o de downtime excesivo en el sistema.
- El fin de concesión se encuentra muy en el futuro y la inyección ya se encuentra madura, por lo que se puede considerar como un estado estacionario.

2. Formalización del MDP

Se procede ahora a explicar la formalización del problema en términos de un MDP, especificando Estados (S), Acciones (A), Transiciones (P) y Recompensas (R). Dadas estas simplificaciones de la realidad, se tiene entonces las dos variables que componen un estado del proceso markoviano de decisión (MDP), watercut (wcut) y el nivel de depleción del pozo (np). El watercut es el porcentaje de agua que se extrae en promedio en un trimestre, de forma que si se extrajeran 100 barriles con un wcut de 0.8, 80 barriles se podrían llenar con agua y sólo 20 con petróleo. wcut varía entre 0,8 y 1 con saltos de 0,01. En el caso de np, es un valor entre 0 y 1000 donde 0 es un pozo del cual no se ha extraído petróleo y 1000 representa un pozo agotado por completo. Los saltos de np son de 10 unidades. Esto hace que la combinación de estados posibles sea de 2121.

$$S = (wcut, np) \forall wcut \in \{0.8, 0.81, \dots, 1\} \wedge np \in \{0, 10, \dots, 1000\}$$

En lo que refiere a las acciones del MDP, como fue explicado previamente, se debe decidir cuánta agua inyectar al pozo, de forma que la frontera de agua formada “empuje” el petróleo hacia el punto

de extracción. Las acciones posibles en este caso son 3 caudales de inyección: High (H), Medium (M) y Low (L). El caudal inyectado determinará la cantidad de fluido a extraer trimestralmente.

$$A \in \{ 'H', 'M', 'L' \}$$

Las transiciones definen cómo se mueve el sistema entre los estados posibles y en este caso definen cómo progresa np (ya que solo puede crecer o mantenerse) y cómo varía el watercut, que solo puede subir en 0.01, mantenerse o bajar en 0.01.

En el caso de np, se define que según la acción que se tome y el wcut resultante hay una probabilidad de mantenerse en el nivel actual y una probabilidad de aumentar en 10 unidades. Esto refleja la complejidad de np en la realidad.

El wcut por su parte depende directamente de la acción que se tome, donde a mayor inyección de agua, más probabilidades hay de que el wcut aumente y cuanto menos agua se inyecte más probabilidades hay de que descienda.

$$P^a(x, y) = (P_{wcut}, P_{np}) \rightarrow wcut' = P_{wcut}(a, wcut); np' = P_{np}(a, np, wcut')$$

Finalmente, en cuanto a la función de recompensa, ésta se basa en un criterio económico de ganancia en la operación del pozo. Los parámetros de este cálculo son el precio del petróleo, el petróleo extraído, el costo fijo de inyectar agua y el costo de disposición de agua.

$$R^a(x, a) = P_{oil} * Q_{oil}(x, a) - C_{iny}(a) - C_{agua}(x, a)$$

donde, P_{oil} es el precio del petróleo que como fue mencionado, se mantendrá fijo durante el ejercicio, Q_{oil} es la cantidad de petróleo extraído que depende del wcut, C_{iny} es el costo de inyección que depende solamente de la acción y C_{agua} es la cantidad de agua por el costo de disposición unitario de conseguir el agua para inyectar, lo cual depende del estado y la acción que se tome. Como punto adicional, la cantidad de fluido que se extrae del pozo depende de la cantidad de agua inyectada (la acción). A su vez, se sabe que ese líquido extraído es una mezcla de agua y petróleo, cuya proporción de agua es wcut y la de petróleo es (1-wcut).

$$Q_{liq} = Q_{liq} * wcut + Q_{liq} * (1 - wcut) = Q_{agua} + Q_{oil}$$

3. Resultados de las políticas obtenidas

Iteración de valor e iteración de política

Se comienza el proceso de solución con el algoritmo de iteración de valor, donde la función *iteracion_de_valor* inicializa todos los estados en 0, para luego ir calculando el valor esperado de cada acción, para cada estado, utilizando la ecuación de Bellman. Una vez obtenidos todos estos posibles valores, se elige la operación que maximice las ganancias esperadas, alcanzando así la política óptima para cada estado.

Continuando con el algoritmo de iteración de política, la función *iteracion_de_politica* comienza con una política fija la cual asigna la acción “H” a todos los estados. A partir de ese camino elegido, se evalúa la política en cada estado a través de la función *evaluar_politica*, y se prosigue por buscar mejorar la política actual para maximizar el valor esperado, a través de la función *mejora_politica*. Surge la necesidad de añadir la función auxiliar *evaluar_politica* para implementar la función de Bellman, ya que la función de *iteracion_de_valor* no evalúa una política fija.

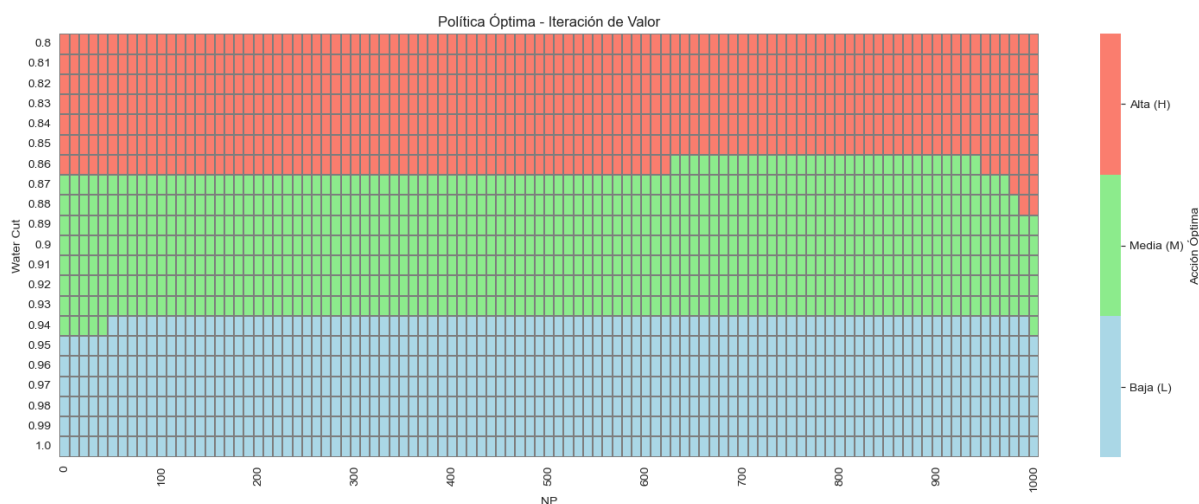
Se analizarán las políticas de estas dos metodologías en conjunto dado que como se utilizaron métodos exactos, el resultado de la política óptima para la iteración de valor es el mismo que para la iteración de política.

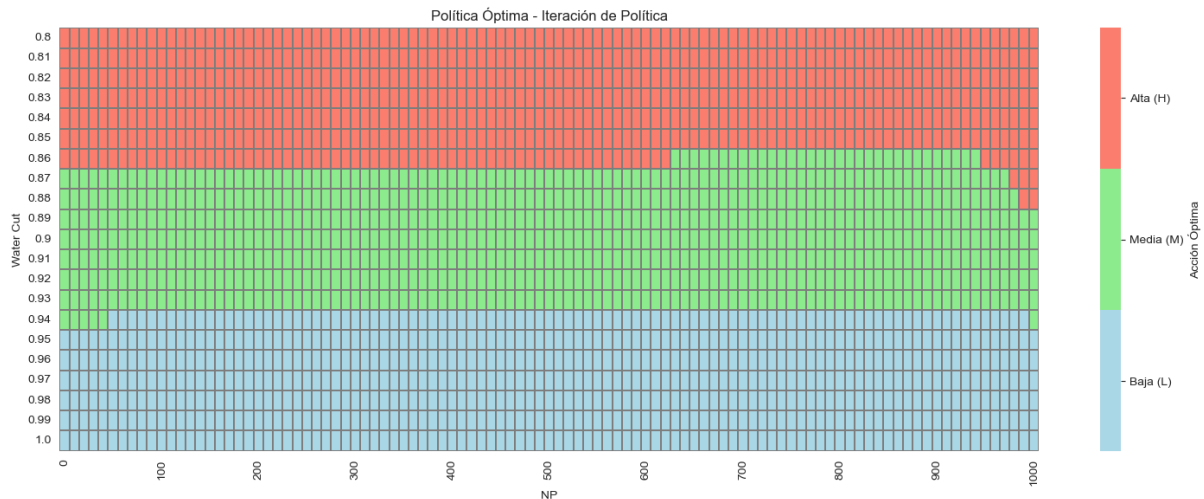
Para las políticas que se muestran a continuación se corrieron escenarios de sensibilidad de cómo variaban con distintos parámetros, especialmente con el caudal de inyección, el caudal producido y la tasa de penalización por depletamiento del reservorio. Solo para un conjunto acotado de parámetros se obtuvo una proporción moderadamente balanceada de las 3 acciones. Se decidió tomar un conjunto de estos parámetros dado que los resultados obtenidos nos resultaron más interesantes para el análisis.

A continuación se muestra un conjunto de parámetros donde la acción M casi no era utilizada en ningún caso:

q_liq_H=120000, c_iny_H=16000: L=1717, M=4, H=400
 q_liq_H=130000, c_iny_H=30000: L=1717, M=2, H=402
 q_liq_H=130000, c_iny_H=25000: L=1717, M=2, H=402
 q_liq_H=130000, c_iny_H=20000: L=1717, M=1, H=403

Con los parámetros finalmente determinados, en las políticas obtenidas por ambos métodos se comienza con una inyección H a bajos w_{cut} y la inyección va bajando a medida que aumenta el corte de agua. La funcionalidad de la política óptima con el Np parece ser bastante débil, observándose sólo variaciones a Np altos, donde se nota una mayor probabilidad de utilizar valores de inyección M que en el resto de la política. **Por qué les parece que pasa eso?**





Otro punto en común de estos dos modelos es el tiempo de ejecución de ambos que es muy comparable, siendo de 30s para iteración por valor y 55s para iteración de política. Dada la similitud entre los tiempos, no pareciera ser una variable determinante entre la elección de uno u otro modelo. ✓

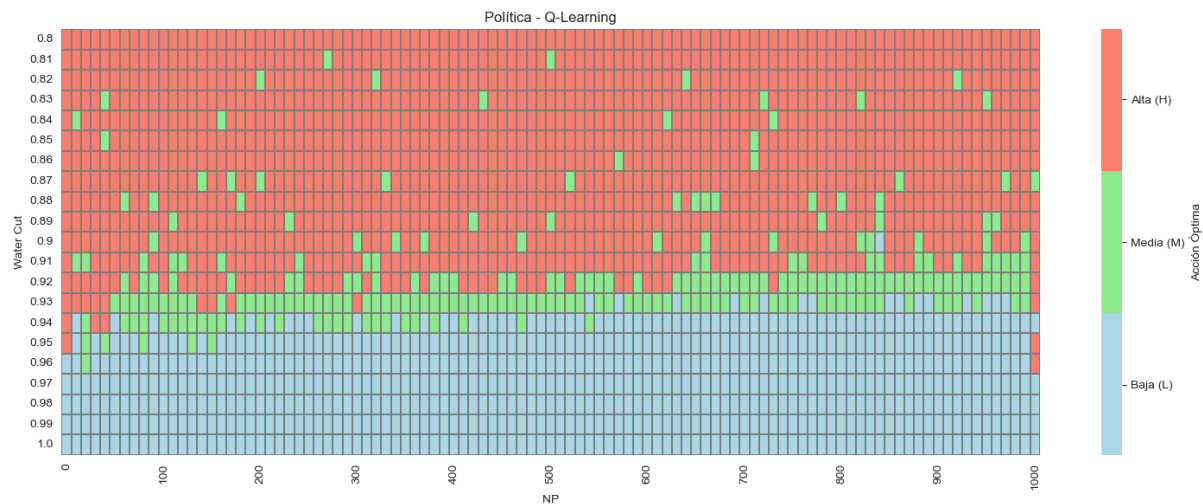
Q-learning

Durante la implementación del algoritmo se evaluaron distintas formas de inicialización del diccionario de valores Q. Inicialmente se utilizó el modo `init='zeros'`, que consiste en iniciar todas las combinaciones de estados y acciones con valores cero, y permitir que sean actualizadas únicamente si son visitadas durante el entrenamiento. Sin embargo, se observó que los resultados no convergían a un óptimo, mostrando resultados erráticos e ilógicos, ya que no se encontraba coherencia en las inyecciones predominantes para múltiples valores de w_{cut} , confirmando una deficiencia en el aprendizaje. Se piensa que este comportamiento del algoritmo está relacionado con que el problema es de maximización y al inicializar en 0 y que las recompensas sean positivas, al usar el mecanismo greedy para decidir por la mejor política, se prioriza la primera acción que se tome en un estado, y dando poco lugar a que una política la reemplace.

En ese caso, se necesita más exploración.

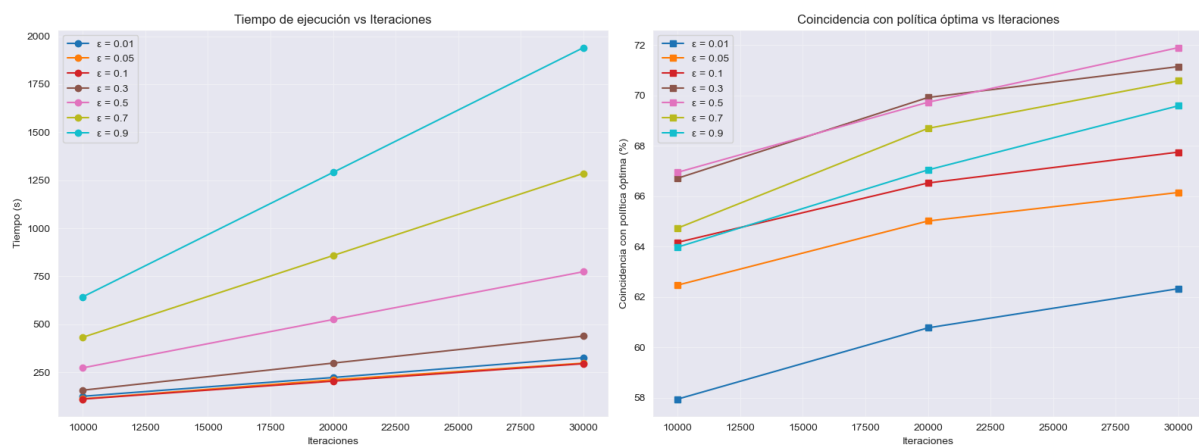


Frente a esta falta de cobertura, se optó por utilizar la opción `init='recorrido'`, que fuerza una pasada inicial por todas las combinaciones de estado y acción antes de comenzar los episodios. De esta manera se asegura que el algoritmo tenga una base mínima de información en todo el espacio de decisiones, lo que permitió obtener una política más completa y con mayor cobertura del dominio. Se entiende que esto no debería distorsionar el funcionamiento del Q-Learning, ya que, como fue mencionado en clase, no importa qué rutas siga el algoritmo ni tampoco qué sean rutas realizables, con lo cual esa “primera pasada” por todos los estados debería ser inocua para el resto del funcionamiento del algoritmo. **Correcto, pero en general necesitan más que una visita a cada par de estado y acción para convergencia (de hecho, infinitas visitas).**



Además de en su inicialización, el modelo se optimizó a partir de una búsqueda del mejor valor para los dos hiperparámetros que dispone, epsilon y la cantidad de iteraciones.

Con respecto a epsilon se corrió sensibilidad para múltiples valores y diferentes cantidad de iteraciones para determinar el valor óptimo para nuestro problema

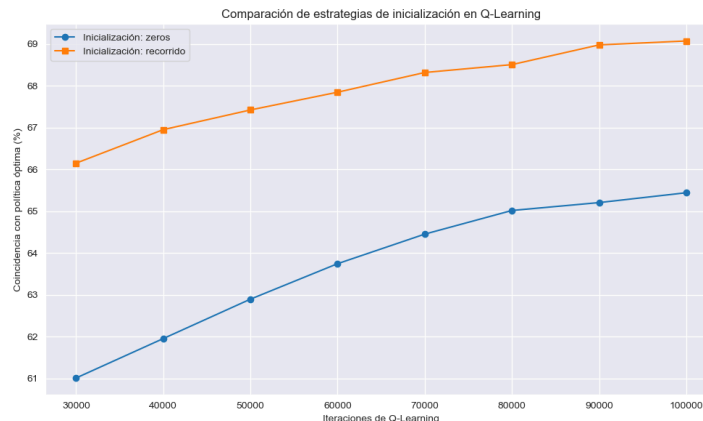


Se observa a partir de los gráficos que el valor de coincidencia con la política óptima mejora a medida que el valor de epsilon aumenta desde 0,01 a 0,5 y luego comienza a descender nuevamente para valores más altos. Esto está significando que el algoritmo aprende mejor en entornos donde hay una combinación de explotación y exploración, aunque tiene mejores resultados con un bias hacia la explotación. Con respecto a los tiempos de ejecución se puede ver que el incremento del tiempo es muy significativo para los dos valores de epsilon más altos.

Compararon la política greedy o la epsilon-greedy con la política óptima? Deberían comparar la política greedy, mismo que el algoritmo de Q-learning esté utilizando la política epsilon-lgreedy.

Dado que para 0,5 el tiempo de ejecución es razonable y obtuvimos la mayor coincidencia con este parámetro va a ser el que vamos a utilizar. Si el tiempo de ejecución fuera un problema, no sería un mal trade off utilizar 0.3, donde se obtiene una performance levemente menor pero en la mitad del tiempo. ✓

Dada la tendencia creciente que se observó en el gráfico anterior de **conciencia vs iteraciones**, se quiso ver si esa tendencia aumentaba indefinidamente o convergía a algún valor, por lo que para un epsilon fijo se aumentó el número de iteraciones hasta 100.000. ?



La mejora que se obtiene por aumentar el número de iteraciones es baja y no pareciera compensar el tiempo adicional necesario en correr el algoritmo, por lo que continuaremos trabajando con 30.000 iteraciones. De hecho, pareciera ser un caso de rendimientos decrecientes y **no está asegurado que con más iteraciones el algoritmo logre converger a la política óptima.**

Con tasa de aprendizaje adecuada y suficientes muestras, el algoritmo tiene que converger.

Teniendo en cuenta la gráfica de la política de Q-Learning hallada se observa que a pesar de contar con una política de exploración balanceada y asegurar la visita a todos los estados, **la política obtenida sigue resultando errática**, especialmente para Wcut bajos. La acción Alta (H) predomina en gran parte del espacio, incluso en regiones donde sería esperable una reducción en el caudal de inyección, lo que sugiere que el algoritmo quedó atrapado en una estrategia subóptima durante el aprendizaje. Incluso luego de extender la cantidad máxima de iteraciones a 100.000 no se observan mejoras significativas en los resultados. Esto sugeriría que, para este entorno donde se conoce el modelo exacto, los algoritmos de solución directa (iteración de valor y política) resultan más efectivos y estables que Q-learning, el cual está más orientado a entornos donde el modelo es desconocido y debe aprenderse a través de la exploración.

Dado los pobres resultados que se estaban obteniendo con la implementación inicial de Q-Learning, se realizaron modificaciones al enfoque con el objetivo de mejorar la calidad de la política aprendida. Se implementó una nueva variante del algoritmo, denominada *q_learning_2*. Esta versión eliminó el esquema ϵ -greedy, optando por seleccionar acciones de manera completamente aleatoria durante todo el proceso de entrenamiento. De esta forma se buscó forzar una exploración más pareja de todo el espacio de decisiones, sin depender de valores Q acumulados tempranamente que pudieran inducir un aprendizaje sesgado.



El resultado fue una política mucho más coherente desde el punto de vista operativo. A diferencia de la versión anterior, *q_learning_2* arrojó una política con un patrón más lógico, priorizando la inyección alta en etapas iniciales y desplazándose hacia decisiones más conservadoras a medida que aumentan el corte de agua y el nivel de depleción. Esta estructura, aunque derivada de acciones puramente aleatorias durante el entrenamiento, capturó mejor la lógica productiva del sistema que la política obtenida por el Q-Learning clásico, obteniéndose un % de coincidencia con la política óptima de un 85%, superando a los valores hallados con la política anterior. ✓

Con respecto a los tiempos de ejecución, estos fueron significativamente más largos que para las iteraciones de valor y política, obteniéndose tiempos de 13m 36s minutos para q-learning inicializado en zeros, 13m 53s para q-learning inicializado en recorrido, y 25m para *q_learning_2*

Cómo eligieron la tasa de aprendizaje? Es otro factor a considerar.

4. Análisis y futuras direcciones

En conclusión, los resultados obtenidos sugieren que en este caso particular, donde se dispone del modelo completo y el entorno es plenamente simulable, los métodos exactos como la iteración de valor y la iteración de política resultan considerablemente más efectivos y estables que los métodos de aprendizaje por refuerzo como Q-learning. La principal debilidad observada en Q-learning radica en su dificultad para balancear adecuadamente exploración y explotación: con pocas iteraciones o una exploración insuficiente, el algoritmo converge rápidamente a políticas subóptimas; **con exploraciones más intensas o forzadas, los tiempos de ejecución crecen de forma significativa sin garantizar mejoras proporcionales en la calidad de las decisiones.**

Eso se puede deber a que bajaron la tasa de aprendizaje demasiado rápido.

La variante *q_learning_2*, basada en una exploración completamente aleatoria, ofreció un resultado sorprendentemente más lógico que su contraparte ϵ -greedy, capturando patrones operativos esperables a lo largo del espacio de estados. Sin embargo, esto se logró a costa de tiempos de cómputo mucho más altos, que no se justifican frente a la robustez y velocidad de los métodos exactos. A futuro, una posible línea de trabajo consiste en estudiar cómo varía la performance de Q-learning en función del número de iteraciones y testear esquemas de exploración más eficientes.

Una de las direcciones que merece un análisis más exhaustivo es el impacto de un horizonte finito en la política óptima de inyección. A modo ilustrativo, se implementó una iteración de valor con horizonte finito de 120 trimestres, correspondiente a un período representativo de la vida

remanente de una concesión madura. El objetivo fue evaluar si las decisiones óptimas varían hacia el final del horizonte, donde las ganancias de largo plazo pierden peso frente a recompensas inmediatas.

Los resultados muestran que la política se mantiene prácticamente constante durante casi todo el horizonte. Se prioriza la inyección baja (L) en estados con alto watercut y NP, mientras que la inyección media (M) aparece en una franja acotada. La inyección alta (H) se selecciona al inicio en la mayoría de los casos con watercut bajo. Solo en los últimos trimestres se observan leves ajustes, extendiendo H a estados que previamente no la contemplaban.

Buena extensión del trabajo original.

